



```
import numpy, pandas
```

```
# arrays for your images, tables for your catalogues
```

```
Maria Vincent | intro2astro | ~8 minutes
```

Maria Vincent

intro2astro.py

numpy.json

# Contents of This Lesson

01 What is numpy

02 Arrays: shape and dtype

03 Picking out pixels

04 Math operations on arrays

05 Statistics along an axis

06 What is pandas

07 Reading and filtering a catalogue

08 Q & A

# class what\_is\_numpy:

*# return definition*

NumPy gives you the ndarray: a grid of numbers, all one type, that you can do maths on all at once. Every astronomical image you open in Python is one of these.

*# it is the layer underneath*

astropy

scipy

matplotlib

photutils

`pip install numpy`                      *# or: conda install numpy*

`import numpy as np`                      **# the alias everyone uses**

*# a loop over a million pixels takes seconds. numpy does it in milliseconds.*

Maria Vincent

intro2astro.py

numpy.txt

# An array has a shape and a dtype

*# these two attributes explain most of your bugs*

```
import numpy as np

img = np.zeros((5, 5))

img.shape    ->  (5, 5)
img.dtype    ->  float64
img.size     ->  25
```

## shape

how big, in each dimension

# (2048, 2048) = a 2k detector

## dtype

what kind of number

# float64, int16, bool

Ways to make one: `np.zeros`, `np.ones`, `np.arange(0, 10, 2)`, `np.linspace(1.0, 2.5, 100)`

Or one falls out of a FITS file: `data = fits.getdata('image.fits') -> numpy.ndarray`

# Picking out pixels: by position

*# a slice says WHERE. rows first, then columns.*

```
img = np.arange(25).reshape(5, 5)      # a tiny 5x5 image
```

**img[0, 0]**

*# one pixel*

|          |    |    |    |    |
|----------|----|----|----|----|
| <b>0</b> | 1  | 2  | 3  | 4  |
| 5        | 6  | 7  | 8  | 9  |
| 10       | 11 | 12 | 13 | 14 |
| 15       | 16 | 17 | 18 | 19 |
| 20       | 21 | 22 | 23 | 24 |

**img[1:4, 1:4]**

*# a 3x3 cutout*

|    |           |           |           |    |
|----|-----------|-----------|-----------|----|
| 0  | 1         | 2         | 3         | 4  |
| 5  | <b>6</b>  | <b>7</b>  | <b>8</b>  | 9  |
| 10 | <b>11</b> | <b>12</b> | <b>13</b> | 14 |
| 15 | <b>16</b> | <b>17</b> | <b>18</b> | 19 |
| 20 | 21        | 22        | 23        | 24 |

**img[:, 2]**

*# a whole column*

|    |    |           |    |    |
|----|----|-----------|----|----|
| 0  | 1  | <b>2</b>  | 3  | 4  |
| 5  | 6  | <b>7</b>  | 8  | 9  |
| 10 | 11 | <b>12</b> | 13 | 14 |
| 15 | 16 | <b>17</b> | 18 | 19 |
| 20 | 21 | <b>22</b> | 23 | 24 |

The colon means "everything". 1:4 means from 1 up to but not including 4.

Careful: a slice is a view of the original, not a copy. Write into it and you change the original array.

# Picking out pixels: by value

*# a mask says WHICH. this is how you handle bad pixels.*

## Step 1: ask a question of the array

```
hot = img > 18  
# an array of True / False,  
# the same shape as img
```

## Step 2: use it to select, or to overwrite

```
img[hot]  
    array([19, 20, ... 24])  
  
img[hot] = np.nan    # flag them
```

the pixels where `img > 18`

|    |    |    |    |    |
|----|----|----|----|----|
| 0  | 1  | 2  | 3  | 4  |
| 5  | 6  | 7  | 8  | 9  |
| 10 | 11 | 12 | 13 | 14 |
| 15 | 16 | 17 | 18 | 19 |
| 20 | 21 | 22 | 23 | 24 |

No loop, no if statement. You describe the pixels you want and numpy finds them.

# Math operations on arrays

*# one line does the whole array. this is the point of numpy.*

```
# arithmetic works elementwise
```

```
img - dark          # subtract a dark
```

```
img / flat          # divide by a flat
```

```
img * 2 + 100
```

```
-2.5 * np.log10(flux)  # to mags
```

```
# and the usual functions
```

```
np.sqrt  np.exp  np.log10
```

```
np.sin   np.abs  np.round
```

```
# these are called ufuncs
```

## Broadcasting: mixing shapes

If one array is smaller, numpy stretches it to fit rather than making you write a loop.

```
img - 100           # one number applied to every pixel
```

```
cube - sky[:, None, None] # one sky value per frame of a data cube
```

# Statistics: what axis= actually means

# 3 exposures, 4 stars, one brightness each

|       | star 1 | star 2 | star 3 | star 4 |
|-------|--------|--------|--------|--------|
| exp 1 | 10     | 12     | 11     | 9      |
| exp 2 | 8      | 15     | 13     | 10     |
| exp 3 | 12     | 11     | 14     | 12     |

→ axis=1

d.shape -> (3, 4)

↓ axis=0

**d.mean(axis=0)**

squash the exposures

-> 4 numbers, one per star

**d.mean(axis=1)**

squash the stars

-> 3 numbers, one per exposure

**d.mean()**

squash everything

-> 1 number

The axis you name is the one that disappears. Same rule for .sum, .std, .max, .median.

Real data has NaNs: use np.nanmean, np.nanstd, np.nanmedian instead.



```
class what_is_pandas:
```

```
    def __init__(self):        return labelled_tables
```

when your data has column names, not just pixel positions

# pandas: a table you can query

*# numpy holds your image. pandas holds your catalogue.*

A DataFrame is a table with named columns and typed rows. Underneath each column is a numpy array, so everything you just learnt still applies.

```
pip install pandas

import pandas as pd

# reads csv, tsv, fixed width,
# excel, and astropy Tables
```

```
cat = pd.read_csv('targets.csv')

cat.head()           # first 5 rows - always look before you compute

cat.columns          # what am I actually working with

cat.describe()       # count, mean, std, min, max of every numeric column
```

# Selecting and filtering a catalogue

*# same idea as a numpy mask, but you name the column*

| name      | ra     | dec    | kmag | filt |
|-----------|--------|--------|------|------|
| V883 Ori  | 83.05  | -7.15  | 8.4  | Ks   |
| BN/KL     | 83.81  | -5.37  | 6.1  | Ks   |
| Fomalhaut | 344.41 | -29.62 | 0.9  | H    |

```
cat['kmag']           # one column
cat[['name', 'kmag']] # two
cat.iloc[0]           # first row
```

## Filter with a condition, exactly like a numpy mask:

```
bright = cat[cat['kmag'] < 7]           # observable tonight
north  = cat[(cat['dec'] > -10) & (cat['kmag'] < 9)] # note the & and the ()

cat.groupby('filt')['kmag'].mean()      # mean magnitude in each filter
cat.sort_values('kmag').head(3)         # your three brightest targets
```

## ASSIGNMENT

# Take a noisy frame and a target list

Mask the hot pixels out of a fake image, then filter a small target list.

```
import numpy as np, pandas as pd
rng = np.random.default_rng(42)
img = rng.normal(100, 5, (64, 64))      # a fake sky frame
img[10, 20] = 5000                      # a hot pixel
```

```
cat = pd.DataFrame({                                # no file needed, the catalogue is right here
    'name': ['V883 Ori', 'BN/KL', 'Fomalhaut', 'HD 214734', 'AB Aur'],
    'ra'   : [83.05, 83.81, 344.41, 226.62, 73.94],
    'dec'  : [-7.15, -5.37, -29.62, 61.30, 30.55],
    'kmag' : [8.4, 6.1, 0.9, 7.2, 4.2],
    'filt' : ['Ks', 'Ks', 'H', 'Ks', 'H'],
})
```

```
# 1. mask everything more than 3 sigma above the median
# 2. print the mean before and after. how much did it move?
# 3. print the targets with dec > -10, then the mean kmag per filter
```

# What I hope you learnt

1 numpy holds your pixels; pandas holds your catalogue

2 shape and dtype tell you what you are holding

3 slices pick by position, masks pick by value

4 maths applies to the whole array, no loop needed

5 the axis you name is the one that disappears

## References:

[numpy.org/doc/stable/user/absolute\\_beginners.html](https://numpy.org/doc/stable/user/absolute_beginners.html)

| [pandas.pydata.org/docs/user\\_guide/10min.html](https://pandas.pydata.org/docs/user_guide/10min.html)

Maria Vincent

intro2astro.py

numpy.txt

```
class questions:
```

```
    def __init__(self):        return answers
```

Do you have any questions?